

Role-Based Authentication (RBAC)

Role-Based Authentication (RBAC) is a system where users are given **roles**, and each role has specific **permissions**.

Instead of giving permissions to every user individually, we:

- Assign a **role** (Admin, User, Manager, etc.)
- Each role defines what actions are allowed
- Users inherit permissions from their role

This makes systems secure, scalable, and easier to manage.

Simple Example

Imagine a system:

Role	Can View	Can Create	Can Delete
Admin	✓	✓	✓
User	✓	✗	✗
Editor	✓	✓	✗

Instead of checking: `if user.username == "admin123":`

We check:

`if user.role == "admin":`

That's RBAC.

How Role-Based Authentication Works (Flow)

1. User registers → role assigned (default: user)
2. User logs in → JWT token generated
3. Token contains user role just like user id
4. Protected route checks role(in jwt token)
5. If role allowed → access granted
6. If not → 403 Forbidden

FastAPI Implementation Plan

Step 1: Add Role Column to Users Table

In user model add:

```
role = Column(String, default="user")
```

Default role → "user"

Admin can be created manually from database.

Step 2: Create Alembic Migration

```
alembic revision --autogenerate -m "add role column"
```

```
alembic upgrade head
```

Step 3: Update User Schema

```
class UserRegister(BaseModel):
    #id :int
    username :str=Field(...,min_length=3)
    email:EmailStr
    password :str=Field(...,min_length=5)

    role: Optional[str] = "user"
```

Step 4: Modify Token Creation

In your create_access_token:access_token = create_access_token(

```
    data={

        "id": d_user.id,

        "username": d_user.username

        "role": user.role

    }

)
```

Step 5: Create role checker function

In security add this:

```
def role_required(required_role: str):
```

```

def role_checker(current_user: User = Depends(get_current_user))

if current_user.role != required_role:

    raise HTTPException(

        status_code=403,

        detail="Access forbidden"

    )

return current_user

return role_checker

```

Step 6: Protect Admin Routes

To give access to user and admin dashboard

Admin— can see all user and his project so make a admin route and make fun like users,projects,tasks

And admin route like

@app.get("/admin")

```

def admin_dashboard(user: User = Depends(role_required("admin"))):
    return {"message": f"Welcome Admin {user.username}"}

```

Step 7: Protect Normal User Routes

User: add in user route user_dashboard that validate user role and show all that which is already implemented for user

@app.get("/profile")

```

def user_profile(

```

```

    user: User = Depends(role_required("user"))

```

```

):

```

```

    return {"message": "Welcome User"}

```

Step 9: Test Cases

1. Register normal user
2. Login → get token
3. Access /admin → should return 403
4. Login as admin
5. Access /admin → should work

Problems:

Every user can role enter himself as an admin

Solution create admin user by pgadmin4 and in registration route ignore role value enter by user or add check that user role cannot be equal to admin

: